Python for Computer Science and Data Science 2 (CSE 3652)

MINOR ASSIGNMENT-5: DEEP LEARNING

Q1. Explain briefly Single Layer Perceptron and Multilayer Perceptron with architecture and illustrate the loss function associated with it.

Single Layer Perceptron (SLP) contains one input layer and one output layer. It is used for linearly separable problems. Multilayer Perceptron (MLP) contains one or more hidden layers between input and output layers and can solve complex nonlinear problems. The loss function measures prediction error. Common loss functions are Mean Squared Error (MSE) and Cross Entropy Loss.

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import mean_squared_error

iris = load_iris()

X = iris.data
y = iris.target

slp = Perceptron()

slp.fit(X, y)

print("Single Layer Perceptron Trained")

mlp = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000
)

mlp.fit(X, y)

print("Multilayer Perceptron Trained")

y_true = [1,2,3]
y_pred = [1.1,1.9,3.2]

loss = mean_squared_error(y_true, y_pred)

print("MSE Loss =", loss)
```

Output:

```
Single Layer Perceptron Trained
Multilayer Perceptron Trained
MSE Loss = 0.020000000000000035
```

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q2. How would you define the architecture of a simple feed-forward ANN for classifying the Iris dataset? Write Python code for the same.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy: 1.0
```

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q3. Build and train a simple ANN using the MNIST dataset to classify handwritten digits. Write Python code for this.

Code:

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

(X_train, y_train), (X_test, y_test) = mnist.load_data()

model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

model.fit(X_train, y_train, epochs=5)
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.8658 - loss: 2.4720
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9192 - loss: 0.3595
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9330 - loss: 0.2691
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9387 - loss: 0.2498
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9463 - loss: 0.2176
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9405 - loss: 0.2500
Accuracy: 0.940500020980835
```

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q4. Find Convolution, ReLU and Max Pooling with the given 4x4 input image and 2x2 filter.

Input Image (4x4): [[1,2,0,1],[3,1,2,2],[1,0,1,3],[2,1,2,1]]

Filter (2x2): [[1,0],[0,-1]]

Code:

```
import numpy as np

image = np.array([
    [1,2,0,1],
    [3,1,2,2],
    [1,0,1,3],
    [2,1,2,1]
])

kernel = np.array([
    [1,0],
    [0,-1]
])

conv = []
```

```
for i in range(3):
    row = []

    for j in range(3):
        value = np.sum(
            image[i:i+2, j:j+2] * kernel
        )

        row.append(value)

    conv.append(row)

conv = np.array(conv)

print("Convolution Output:")
print(conv)

relu = np.maximum(0, conv)

print("\nReLU Output:")
print(relu)

max_pool = np.max(relu)

print("\nMax Pooling Output:")
print([[max_pool]])
```

Output:

```
Convolution Output:
[[ 0  0 -2]
 [ 3  0 -1]
 [ 0 -2  0]]

ReLU Output:
[[0 0 0]
 [3 0 0]
 [0 0 0]]

Max Pooling Output:
[[np.int64(3)]]
```


Q5. Build a CNN with two convolutional layers and one fully connected hidden layer to classify handwritten digits from the MNIST dataset.

Code:

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train.reshape(-1,28,28,1) / 255.0
X_test = X_test.reshape(-1,28,28,1) / 255.0

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model.fit(X_train, y_train, epochs=5)
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

Output:

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 71s 36ms/step - accuracy: 0.9592 - loss: 0.1305
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 59s 31ms/step - accuracy: 0.9866 - loss: 0.0427
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.9905 - loss: 0.0298
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.9935 - loss: 0.0206
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 30ms/step - accuracy: 0.9949 - loss: 0.0162
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9909 - loss: 0.0310
Accuracy: 0.9908999800682068
```